

Backtesting avec QuantConnect LEAN

Table of Contents

Backtesting de stratégies de trading avec QuantConnect LEAN.....	1
Paramètres communs à tous les exemples.....	2
1. Présentation de LEAN	2
2. Installation et mise en place	3
3. Concepts fondamentaux : moteur événementiel.....	3
Exemple 1 — Buy & Hold	4
Exemple 2 — Croisement de moyennes mobiles (SMA 50/200)	5
Exemple 3 — Retour à la moyenne (RSI 14).....	6
Exemple 4 — Bandes de Bollinger (20/2).....	7
Exemple 5 — MACD (12/26/9)	8
Exemple 6 — Portefeuille multi-actifs	9
Exemple 7 — Dimensionnement de position et risque	10
Exemple 8 — Stop-loss et take-profit.....	11
Exemple 9 — Analyse des métriques de performance	13
Exemple 10 — Optimisation de paramètres.....	14
Exemple 11 — Validation walk-forward / hors-échantillon	15
Tableau comparatif : LEAN vs Vectorbt.....	17
Exemple 12 — Stratégie avancée complète	18
15. Conclusion.....	21

Backtesting de stratégies de trading avec QuantConnect LEAN

Document formatif — Plateforme **QuantConnect LEAN** (API Python, syntaxe PEP8 / snake_case).

Ce document est le jumeau de celui consacré à **Vectorbt / Vectorbt Pro** : il contient **exactement les mêmes exemples**, mais transposés aux idiomes de LEAN.

Paramètres communs à tous les exemples

Pour que les exemples soient strictement comparables d'une plateforme à l'autre, ils partagent tous la même configuration :

Paramètre	Valeur
Capital initial	100 000 \$
Frais de transaction	0,1 % (0,001)
Période de backtest	2020-01-01 → 2023-12-31
Actif principal	SPY
Panier multi-actifs	AAPL, MSFT, GOOG
Résolution	Journalière (Resolution.DAILY)
SMA croisement	rapide 50 / lente 200
RSI	période 14, seuils 30 / 70
Bandes de Bollinger	période 20, écart-type 2
MACD	12 / 26 / 9

1. Présentation de LEAN

LEAN est le moteur de backtesting et de trading algorithmique open-source qui propulse la plateforme **QuantConnect**. Écrit en C#, il s'utilise indifféremment en C# ou en Python (via Python.NET). Ce document utilise l'API Python moderne en **snake_case** (norme PEP8).

Points caractéristiques :

- **Moteur événementiel** : votre code reçoit les données barre par barre, dans l'ordre chronologique, exactement comme en trading réel. Cela élimine par construction la *look-ahead bias* (utilisation d'information future).
- **Modélisation réaliste** : frais, slippage, marges, dividendes, splits, heures de marché étendues sont modélisés.
- **Du backtest au live** : le même code peut passer en trading réel sans réécriture.
- **Données intégrées** : actions US, options, futures, forex, crypto, données fondamentales et alternatives.

Toute stratégie hérite de la classe `QCAAlgorithm` et redéfinit principalement deux méthodes : `initialize` (configuration) et `on_data` (logique de trading appelée à chaque nouvelle barre).

2. Installation et mise en place

Trois manières de travailler avec LEAN :

a) Plateforme web QuantConnect (la plus simple) — Aucune installation : on écrit le code dans l'IDE en ligne et on lance le backtest. Idéal pour débiter.

b) LEAN en local via la CLI

```
# Installation de la CLI (nécessite Python 3.8+ et Docker)
pip install lean

# Connexion à votre compte QuantConnect
lean login

# Initialiser un répertoire de travail
lean init

# Créer un nouveau projet
lean project-create "Ma Strategie" --language python

# Lancer un backtest local (Le moteur tourne dans un conteneur Docker)
lean backtest "Ma Strategie"
```

c) LEAN open-source depuis GitHub — Cloner QuantConnect/Lean et l'exécuter avec .NET pour un contrôle total sur l'infrastructure.

Tout fichier de stratégie Python commence par l'import global qui expose l'ensemble des types LEAN :

```
from AlgorithmImports import *
```

3. Concepts fondamentaux : moteur événementiel

LEAN est **événementiel** (event-driven), contrairement à une approche vectorisée. Comprendre cette différence est essentiel.

```
from AlgorithmImports import *

class StructureDeBaseAlgorithm(QCAlgorithm):

    def initialize(self):
```

```

# 1) Période et capital
self.set_start_date(2020, 1, 1)
self.set_end_date(2023, 12, 31)
self.set_cash(100_000)

# 2) Souscription aux données (résolution journalière)
self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol

# 3) Modèle de frais : 0,1 % par transaction
self.set_brokerage_model(BrokerageName.INTERACTIVE_BROKERS_BROKERAGE)

def on_data(self, slice: Slice):
    # Appelée à chaque nouvelle barre, dans l'ordre chronologique.
    # On vérifie toujours que la donnée du symbole est présente.
    if self._symbol not in slice.bars:
        return

    prix = slice.bars[self._symbol].close
    # ... Logique de décision ici ...

```

Idées-clés :

- initialize ne s'exécute **qu'une fois**, au démarrage.
- on_data s'exécute **à chaque barre** : c'est la boucle de simulation. À l'instant t , vous ne voyez **jamais** les prix futurs.
- Les **indicateurs** sont mis à jour automatiquement par le moteur ; on attend qu'ils soient « prêts » (is_ready) et que la phase de préchauffage (is_warming_up) soit terminée avant de trader.
- Les **ordres** se passent via des méthodes comme set_holdings, market_order ou liquidate.

Exemple 1 — Buy & Hold

Objectif pédagogique : la stratégie de référence. On achète SPY au premier jour et on conserve jusqu'à la fin. Toute stratégie active devra battre ce *benchmark*.

```

from AlgorithmImports import *

class BuyAndHoldAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2020, 1, 1)

```

```

self.set_end_date(2023, 12, 31)
self.set_cash(100_000)

self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
# Frais de 0,1 % par transaction
self.securities[self._symbol].set_fee_model(ConstantFeeModel(0))
self.set_brokerage_model(BrokerageName.INTERACTIVE_BROKERS_BROKERAGE)

def on_data(self, slice: Slice):
    if self._symbol not in slice.bars:
        return
    # On investit 100 % du capital une seule fois, puis on ne touche plus.
    if not self.portfolio.invested:
        self.set_holdings(self._symbol, 1.0)

```

À retenir : `self.portfolio.invested` est vrai dès qu'au moins une position est ouverte ; le `if not ... invested` garantit qu'on n'achète qu'une fois. LEAN calcule automatiquement le rendement total, le ratio de Sharpe et le drawdown maximum dans le rapport de backtest.

Exemple 2 — Croisement de moyennes mobiles (SMA 50/200)

Objectif pédagogique : la stratégie de suivi de tendance la plus classique (*golden cross / death cross*). On est long quand la SMA 50 passe au-dessus de la SMA 200, on liquide au croisement inverse.

```

from AlgorithmImports import *

class SmaCrossoverAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2020, 1, 1)
        self.set_end_date(2023, 12, 31)
        self.set_cash(100_000)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol

        # Deux moyennes mobiles simples, mises à jour automatiquement par Le moteur.
        self._fast = self.sma(self._symbol, 50, Resolution.DAILY)
        self._slow = self.sma(self._symbol, 200, Resolution.DAILY)

        # Préchauffage : on attend 200 barres pour que La SMA lente soit valide.

```

```

self.set_warm_up(200, Resolution.DAILY)

def on_data(self, slice: Slice):
    # On ne trade pas tant que les indicateurs ne sont pas prêts.
    if self.is_warming_up or not self._slow.is_ready:
        return
    if self._symbol not in slice.bars:
        return

    # Entrée : croisement haussier (SMA rapide > SMA lente)
    if self._fast.current.value > self._slow.current.value:
        if not self.portfolio[self._symbol].invested:
            self.set_holdings(self._symbol, 1.0)
    # Sortie : croisement baissier
    else:
        if self.portfolio[self._symbol].invested:
            self.liquidate(self._symbol)

```

À retenir : `self.sma(...)` est une méthode d'aide qui crée et branche l'indicateur sur le flux de données. On lit sa valeur courante avec `.current.value`. Le `set_warm_up` évite de prendre des décisions sur un indicateur encore incomplet.

Exemple 3 — Retour à la moyenne (RSI 14)

Objectif pédagogique : stratégie contrarienne. On achète en survente ($RSI < 30$) et on vend en surachat ($RSI > 70$).

```

from AlgorithmImports import *

class RsiMeanReversionAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2020, 1, 1)
        self.set_end_date(2023, 12, 31)
        self.set_cash(100_000)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol

        # RSI sur 14 périodes.
        self._rsi = self.rsi(self._symbol, 14, MovingAverageType.WILDERS, Resolution.DAILY)
        self.set_warm_up(14, Resolution.DAILY)

```

```

def on_data(self, slice: Slice):
    if self.is_warming_up or not self._rsi.is_ready:
        return
    if self._symbol not in slice.bars:
        return

    valeur = self._rsi.current.value

    # Achat en survente
    if valeur < 30 and not self.portfolio[self._symbol].invested:
        self.set_holdings(self._symbol, 1.0)
    # Vente en surachat
    elif valeur > 70 and self.portfolio[self._symbol].invested:
        self.liquidate(self._symbol)

```

À retenir : on choisit ici la méthode de lissage de Wilder (MovingAverageType.WILDERS), qui est le calcul canonique du RSI. La logique « acheter bas / vendre haut » est l'inverse du suivi de tendance.

Exemple 4 — Bandes de Bollinger (20/2)

Objectif pédagogique : retour à la moyenne sur volatilité. On achète quand le prix touche la bande inférieure, on vend quand il revient sur la bande médiane (ou supérieure).

```

from AlgorithmImports import *

class BollingerBandsAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2020, 1, 1)
        self.set_end_date(2023, 12, 31)
        self.set_cash(100_000)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol

        # Bandes de Bollinger : période 20, 2 écarts-types.
        self._bb = self.bb(self._symbol, 20, 2, MovingAverageType.SIMPLE, Resolution.DAILY)
        self.set_warm_up(20, Resolution.DAILY)

    def on_data(self, slice: Slice):
        if self.is_warming_up or not self._bb.is_ready:

```

```

    return
    if self._symbol not in slice.bars:
        return

    prix = slice.bars[self._symbol].close
    bande_basse = self._bb.lower_band.current.value
    bande_milieu = self._bb.middle_band.current.value

    # Entrée : Le prix franchit la bande inférieure (survente)
    if prix < bande_basse and not self.portfolio[self._symbol].invested:
        self.set_holdings(self._symbol, 1.0)
    # Sortie : Le prix repasse au-dessus de la bande médiane
    elif prix > bande_milieu and self.portfolio[self._symbol].invested:
        self.liquidate(self._symbol)

```

À retenir : l'indicateur bb expose trois sous-indicateurs : lower_band, middle_band, upper_band. On compare le prix de clôture à ces niveaux.

Exemple 5 — MACD (12/26/9)

Objectif pédagogique : suivi de tendance via le MACD. On est long quand la ligne MACD passe au-dessus de sa ligne de signal.

```

from AlgorithmImports import *

class MacdAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2020, 1, 1)
        self.set_end_date(2023, 12, 31)
        self.set_cash(100_000)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol

        # MACD : EMA rapide 12, EMA lente 26, signal 9.
        self._macd = self.macd(self._symbol, 12, 26, 9, MovingAverageType.EXPONENTIAL, Resolution.DAILY)
        self.set_warm_up(35, Resolution.DAILY)

    def on_data(self, slice: Slice):
        if self.is_warming_up or not self._macd.is_ready:
            return

```



```

if self._symbol not in slice.bars:
    return

ligne_macd = self._macd.current.value
ligne_signal = self._macd.signal.current.value

# Entrée : MACD au-dessus du signal (momentum haussier)
if ligne_macd > ligne_signal and not self.portfolio[self._symbol].invested:
    self.set_holdings(self._symbol, 1.0)
# Sortie : MACD repasse sous le signal
elif ligne_macd < ligne_signal and self.portfolio[self._symbol].invested:
    self.liquidate(self._symbol)

```

À retenir : `self._macd.current.value` est la ligne MACD elle-même ; `self._macd.signal.current.value` est la ligne de signal. Le croisement de ces deux lignes déclenche les ordres.

Exemple 6 — Portefeuille multi-actifs

Objectif pédagogique : appliquer la même stratégie (croisement SMA 50/200) simultanément à plusieurs titres (AAPL, MSFT, GOOG), avec une allocation égale ($\approx 33\%$ par titre quand le signal est actif).

```

from AlgorithmImports import *

class MultiAssetSmaAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2020, 1, 1)
        self.set_end_date(2023, 12, 31)
        self.set_cash(100_000)

        self._tickers = ["AAPL", "MSFT", "GOOG"]
        self._indicateurs = {}

        for ticker in self._tickers:
            symbol = self.add_equity(ticker, Resolution.DAILY).symbol
            self._indicateurs[symbol] = {
                "fast": self.sma(symbol, 50, Resolution.DAILY),
                "slow": self.sma(symbol, 200, Resolution.DAILY),
            }

```

```

self.set_warm_up(200, Resolution.DAILY)

def on_data(self, slice: Slice):
    if self.is_warming_up:
        return

    # Allocation cible par titre quand le signal est actif.
    poids_cible = 1.0 / len(self._tickers)

    for symbol, ind in self._indicateurs.items():
        if not ind["slow"].is_ready or symbol not in slice.bars:
            continue

        signal_haussier = ind["fast"].current.value > ind["slow"].current.value

        if signal_haussier and not self.portfolio[symbol].invested:
            self.set_holdings(symbol, poids_cible)
        elif not signal_haussier and self.portfolio[symbol].invested:
            self.liquidate(symbol)

```

À retenir : on stocke un jeu d'indicateurs par symbole dans un dictionnaire. `set_holdings(symbol, 0.33)` demande au moteur d'allouer 33 % de la valeur du portefeuille à ce titre — LEAN calcule lui-même la quantité d'actions à acheter ou vendre.

Exemple 7 — Dimensionnement de position et risque

Objectif pédagogique : ne pas investir 100 % à chaque signal. Ici, on n'engage que **95 % du capital** par entrée (en gardant un coussin de liquidités) sur la stratégie de croisement SMA.

```

from AlgorithmImports import *

class PositionSizingAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2020, 1, 1)
        self.set_end_date(2023, 12, 31)
        self.set_cash(100_000)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._fast = self.sma(self._symbol, 50, Resolution.DAILY)
        self._slow = self.sma(self._symbol, 200, Resolution.DAILY)

```

```

self.set_warm_up(200, Resolution.DAILY)

# Fraction du capital engagée à chaque trade.
self._taille_position = 0.95

def on_data(self, slice: Slice):
    if self.is_warming_up or not self._slow.is_ready:
        return
    if self._symbol not in slice.bars:
        return

    if self._fast.current.value > self._slow.current.value:
        if not self.portfolio[self._symbol].invested:
            # On n'engage que 95 % du capital.
            self.set_holdings(self._symbol, self._taille_position)
        else:
            if self.portfolio[self._symbol].invested:
                self.liquidate(self._symbol)

```

À retenir : `set_holdings(symbol, 0.95)` plafonne l'exposition à 95 %. Le dimensionnement de position est un levier de gestion du risque au moins aussi important que les signaux d'entrée/sortie.

Exemple 8 — Stop-loss et take-profit

Objectif pédagogique : ajouter une sortie en perte (stop-loss à 5 % sous le prix d'entrée) et une prise de bénéfice (take-profit à 10 % au-dessus), greffées sur la stratégie de croisement SMA.

```

from AlgorithmImports import *

class StopLossTakeProfitAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2020, 1, 1)
        self.set_end_date(2023, 12, 31)
        self.set_cash(100_000)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._fast = self.sma(self._symbol, 50, Resolution.DAILY)
        self._slow = self.sma(self._symbol, 200, Resolution.DAILY)
        self.set_warm_up(200, Resolution.DAILY)

```

```

self._prix_entree = None
self._stop_pct = 0.05    # 5 %
self._tp_pct = 0.10     # 10 %

def on_data(self, slice: Slice):
    if self.is_warming_up or not self._slow.is_ready:
        return
    if self._symbol not in slice.bars:
        return

    prix = slice.bars[self._symbol].close
    investi = self.portfolio[self._symbol].invested

    # Gestion des sorties stop / TP quand on est en position.
    if investi and self._prix_entree is not None:
        if prix <= self._prix_entree * (1 - self._stop_pct):
            self.liquidate(self._symbol)    # stop-loss touché
            self._prix_entree = None
            return
        if prix >= self._prix_entree * (1 + self._tp_pct):
            self.liquidate(self._symbol)    # take-profit touché
            self._prix_entree = None
            return

    signal_haussier = self._fast.current.value > self._slow.current.value

    # Entrée
    if signal_haussier and not investi:
        self.set_holdings(self._symbol, 1.0)
        self._prix_entree = prix
    # Sortie sur signal baissier
    elif not signal_haussier and investi:
        self.liquidate(self._symbol)
        self._prix_entree = None

```

À retenir : on mémorise `self._prix_entree` à l'achat et on le compare au prix courant à chaque barre. En production, on peut aussi passer de vrais ordres `stop_market_order` et `limit_order` pour que la sortie soit gérée côté courtier, mais le suivi manuel est plus transparent pour l'apprentissage.

Exemple 9 — Analyse des métriques de performance

Objectif pédagogique : ne pas se contenter du rendement total. On veut le ratio de Sharpe, le drawdown maximum, le taux de réussite, etc.

Dans LEAN, le rapport de backtest **calcule automatiquement** l'essentiel de ces métriques (Sharpe Ratio, Maximum Drawdown, Total Trades, Win Rate, Probabilistic Sharpe Ratio...). On peut aussi les consigner manuellement à la fin de l'exécution :

```
from AlgorithmImports import *

class PerformanceMetricsAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2020, 1, 1)
        self.set_end_date(2023, 12, 31)
        self.set_cash(100_000)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol
        self._fast = self.sma(self._symbol, 50, Resolution.DAILY)
        self._slow = self.sma(self._symbol, 200, Resolution.DAILY)
        self.set_warm_up(200, Resolution.DAILY)

        # On suit la valeur du portefeuille pour calculer nos propres métriques.
        self._valeurs = []

    def on_data(self, slice: Slice):
        # Enregistre la valeur totale du portefeuille à chaque barre.
        self._valeurs.append(self.portfolio.total_portfolio_value)

        if self.is_warming_up or not self._slow.is_ready:
            return
        if self._symbol not in slice.bars:
            return

        if self._fast.current.value > self._slow.current.value:
            if not self.portfolio[self._symbol].invested:
                self.set_holdings(self._symbol, 1.0)
        else:
            if self.portfolio[self._symbol].invested:
                self.liquidate(self._symbol)

    def on_end_of_algorithm(self):
```

```

# Métriques personnalisées calculées en fin de backtest.
import numpy as np
valeurs = np.array(self._valeurs, dtype=float)
rendements = np.diff(valeurs) / valeurs[:-1]

rendement_total = valeurs[-1] / valeurs[0] - 1
# Sharpe annualisé (252 jours de bourse), taux sans risque supposé nul.
sharpe = np.sqrt(252) * rendements.mean() / rendements.std()
# Drawdown maximum
pics = np.maximum.accumulate(valeurs)
drawdown_max = ((valeurs - pics) / pics).min()

self.log(f"Rendement total      : {rendement_total:.2%}")
self.log(f"Ratio de Sharpe      : {sharpe:.2f}")
self.log(f"Drawdown maximum      : {drawdown_max:.2%}")
self.log(f"Valeur finale          : {valeurs[-1]:.0f} $")

```

À retenir : `on_end_of_algorithm` est appelée une fois à la fin du backtest. C'est l'endroit idéal pour récapituler des statistiques. Le tableau de statistiques officiel de LEAN reste néanmoins la référence — comparez toujours votre Sharpe à celui d'un simple Buy & Hold.

Exemple 10 — Optimisation de paramètres

Objectif pédagogique : trouver le meilleur couple (SMA rapide, SMA lente) parmi une grille — rapide $\in \{10, 20, 50\}$, lente $\in \{100, 150, 200\}$.

LEAN est événementiel : il **n'essaie pas** toutes les combinaisons en une seule passe (contrairement à un moteur vectorisé). On expose les paramètres via `get_parameter`, et l'**outil d'optimisation** de QuantConnect relance le backtest pour chaque combinaison.

```

from AlgorithmImports import *

class ParameterOptimizationAlgorithm(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2020, 1, 1)
        self.set_end_date(2023, 12, 31)
        self.set_cash(100_000)

        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol

        # Les paramètres sont injectés par l'optimiseur ; valeurs par défaut sinon.

```

```

fast_period = self.get_parameter("fast_period", 50)
slow_period = self.get_parameter("slow_period", 200)

self._fast = self.sma(self._symbol, fast_period, Resolution.DAILY)
self._slow = self.sma(self._symbol, slow_period, Resolution.DAILY)
self.set_warm_up(slow_period, Resolution.DAILY)

def on_data(self, slice: Slice):
    if self.is_warming_up or not self._slow.is_ready:
        return
    if self._symbol not in slice.bars:
        return

    if self._fast.current.value > self._slow.current.value:
        if not self.portfolio[self._symbol].invested:
            self.set_holdings(self._symbol, 1.0)
    else:
        if self.portfolio[self._symbol].invested:
            self.liquidate(self._symbol)

```

Côté configuration de l'optimisation (interface QuantConnect ou fichier config.json en local), on déclare la grille et la cible :

```

{
  "parameters": {
    "fast_period": { "min": 10, "max": 50, "step": 20 },
    "slow_period": { "min": 100, "max": 200, "step": 50 }
  },
  "objective": "SharpeRatio",
  "target": "max"
}

```

À retenir : `self.get_parameter("nom", défaut)` rend la stratégie paramétrable sans modifier le code. L'optimiseur de LEAN exécute un backtest complet par combinaison et classe les résultats selon l'objectif (ici, maximiser le Sharpe). **Attention au sur-optimisation (*overfitting*)** : une grille trop fine donne des paramètres « parfaits » sur le passé mais fragiles en réel.

Exemple 11 — Validation walk-forward / hors-échantillon

Objectif pédagogique : valider la robustesse en séparant la période d'**entraînement** (in-sample, où l'on optimise) de la période de **test** (out-of-sample, jamais vue). On optimise sur 2020-2022 et on valide sur 2023.

L'approche idiomatique en LEAN : on définit une période **hors-échantillon** que l'organisation peut imposer, et on lance deux backtests distincts (un sur l'in-sample, un sur l'out-of-sample) avec les paramètres figés trouvés à l'étape d'optimisation.

```
from AlgorithmImports import *

class WalkForwardAlgorithm(QCAlgorithm):

    def initialize(self):
        # Phase pilotée par paramètre : "train" (2020-2022) ou "test" (2023).
        phase = self.get_parameter("phase", "test")

        if phase == "train":
            self.set_start_date(2020, 1, 1)
            self.set_end_date(2022, 12, 31) # in-sample (optimisation)
        else:
            self.set_start_date(2023, 1, 1)
            self.set_end_date(2023, 12, 31) # out-of-sample (validation)

        self.set_cash(100_000)
        self._symbol = self.add_equity("SPY", Resolution.DAILY).symbol

        # Paramètres figés issus de l'optimisation in-sample.
        fast_period = self.get_parameter("fast_period", 50)
        slow_period = self.get_parameter("slow_period", 200)
        self._fast = self.sma(self._symbol, fast_period, Resolution.DAILY)
        self._slow = self.sma(self._symbol, slow_period, Resolution.DAILY)
        self.set_warm_up(slow_period, Resolution.DAILY)

    def on_data(self, slice: Slice):
        if self.is_warming_up or not self._slow.is_ready:
            return
        if self._symbol not in slice.bars:
            return

        if self._fast.current.value > self._slow.current.value:
            if not self.portfolio[self._symbol].invested:
                self.set_holdings(self._symbol, 1.0)
        else:
            if self.portfolio[self._symbol].invested:
                self.liquidate(self._symbol)
```


Protocole :

1. **Phase train** : on lance l'optimiseur (Exemple 10) **uniquement** sur 2020-2022.
 2. On retient le meilleur couple (`fast_period`, `slow_period`).
 3. **Phase test** : on relance ce seul jeu de paramètres sur 2023.
 4. On compare la performance in-sample et out-of-sample : si elle s'effondre hors-échantillon, la stratégie est sur-optimisée.
- À retenir : LEAN n'a pas de fonction « walk-forward » en une ligne ; on l'organise par périodes paramétrées. La discipline in-sample / out-of-sample est la protection principale contre l'illusion de performance.

Tableau comparatif : LEAN vs Vectorbt

Les deux plateformes répondent au même besoin — backtester des stratégies en Python — mais selon deux philosophies opposées. Le tableau ci-dessous synthétise leurs points communs et leurs différences.

Similitudes

Point commun	Détail
Objectif	Backtester des stratégies de trading sur données historiques
Langage	Python (LEAN propose aussi C#)
Indicateurs intégrés	SMA, EMA, RSI, Bandes de Bollinger, MACD, ATR, etc.
Logique entrée/sortie	Signaux d'achat et de vente, gestion de positions
Gestion du risque	Stop-loss, take-profit, dimensionnement de position
Frais	Modélisation des coûts de transaction
Métriques	Rendement, Sharpe, drawdown, taux de réussite, etc.
Optimisation	Recherche de paramètres et validation hors-échantillon
Licence	Code source ouvert (LEAN entièrement ; Vectorbt en open-source, Pro commercial)

Différences

Aspect	QuantConnect LEAN	Vectorbt / Vectorbt Pro
Paradigme	Événementiel (barre par barre)	Vectorisé (toute la série d'un coup)
Langages	Python et C#	Python uniquement
Vitesse de backtest	Plus lente (simulation séquentielle)	Très rapide (NumPy + Numba)
Exploration de paramètres	Un backtest relancé par combinaison	Des milliers de combinaisons en une passe
Réalisme de la microstructure	Élevé (slippage, marge, fills, dividendes, splits)	Simplifié

Aspect	QuantConnect LEAN	Vectorbt / Vectorbt Pro
Look-ahead bias	Impossible par construction	Possible si le code est mal écrit
Passage au trading réel	Natif : le même code passe en live	Surtout recherche/backtest ; live limité
Données	Marketplace intégrée + connecteurs courtiers	Sources externes (yfinance, CSV, API...)
Contrôle fin par ordre	Granulaire et natif	Via callbacks (surtout en Pro)
Courbe d'apprentissage	Cadre structuré QCAgorithm	API pandas/NumPy familière
Coût	Cloud freemium ; moteur gratuit en local	Vectorbt gratuit ; Pro payant

Avantages et inconvénients

	QuantConnect LEAN	Vectorbt / Vectorbt Pro
Avantages	Réalisme élevé ; pas de look-ahead ; passage au live sans réécriture ; données et classes d'actifs intégrées ; gestion fine des ordres	Vitesse extrême ; exploration massive de paramètres ; analyse statistique riche (stats()) ; idéal pour la recherche et le machine learning
Inconvénients	Backtests plus lents ; optimisation coûteuse en calcul ; code plus verbeux ; dépendance à Docker / plateforme en local	Risque de look-ahead ; modélisation moins réaliste ; pas de live natif robuste ; logique par-ordre plus complexe ; fonctions avancées payantes (Pro)

En résumé : on choisit **LEAN** pour la fidélité au marché réel et le passage en production, et **Vectorbt** pour la rapidité de recherche et l'exploration de très nombreuses idées. Les deux sont complémentaires : prototyper et filtrer des idées sous Vectorbt, puis valider et déployer la meilleure sous LEAN, est un flux de travail courant.

Exemple 12 — Stratégie avancée complète

Objectif pédagogique : réunir en une seule stratégie la plupart des concepts précédents. Il s'agit d'une **stratégie multi-actifs de suivi de tendance, filtrée par le régime de marché et pilotée par le risque** :

- **Univers** : AAPL, MSFT, GOOG.
- **Filtre de régime** : on ne prend position que si le prix est au-dessus de sa SMA 200 (tendance de fond haussière).
- **Signal d'entrée** : confirmation de momentum — ligne MACD au-dessus de sa ligne de signal **et** $RSI(14) > 50$.
- **Dimensionnement par le risque** : on risque **1 % du capital** par position ; la taille se déduit de la distance au stop (volatilité mesurée par l'ATR).
- **Stop suiveur** : $2 \times ATR(14)$ sous le plus haut atteint depuis l'entrée.
- **Take-profit** : $4 \times ATR$ au-dessus de l'entrée (ratio risque/rendement 2:1).
- **Limite d'exposition** : 3 positions simultanées au maximum.

```

from AlgorithmImports import *

class AdvancedRegimeStrategy(QCAlgorithm):

    def initialize(self):
        self.set_start_date(2020, 1, 1)
        self.set_end_date(2023, 12, 31)
        self.set_cash(100_000)

        # Paramètres de risque
        self._risk_per_trade = 0.01      # 1 % du capital risqué par position
        self._atr_stop_mult = 2.0        # stop à 2 x ATR
        self._atr_tp_mult = 4.0          # take-profit à 4 x ATR (ratio 2:1)
        self._max_positions = 3

        # Un jeu d'indicateurs + état par actif.
        self._tickers = ["AAPL", "MSFT", "GOOG"]
        self._data = {}
        for t in self._tickers:
            symbol = self.add_equity(t, Resolution.DAILY).symbol
            self._data[symbol] = {
                "trend": self.sma(symbol, 200, Resolution.DAILY),
                "macd": self.macd(symbol, 12, 26, 9,
                                   MovingAverageType.EXPONENTIAL, Resolution.DAILY),
                "rsi": self.rsi(symbol, 14, MovingAverageType.WILDERS, Resolution.DAILY),
                "atr": self.atr(symbol, 14, MovingAverageType.WILDERS, Resolution.DAILY),
                "entry_price": None,
                "stop_price": None,
            }

        self.set_warm_up(200, Resolution.DAILY)

    def on_data(self, slice: Slice):
        if self.is_warming_up:
            return

        for symbol, d in self._data.items():
            # Tous les indicateurs doivent être prêts et la barre présente.
            if not all(d[k].is_ready for k in ("trend", "macd", "rsi", "atr")):
                continue
            if symbol not in slice.bars:

```

continue

```
prix = slice.bars[symbol].close
atr = d["atr"].current.value
investi = self.portfolio[symbol].invested

# --- Gestion des sorties : stop suiveur et take-profit ---
if investi:
    # Le stop ne fait que monter (stop suiveur).
    nouveau_stop = prix - self._atr_stop_mult * atr
    if d["stop_price"] is None or nouveau_stop > d["stop_price"]:
        d["stop_price"] = nouveau_stop

    stop_touche = prix <= d["stop_price"]
    tp_touche = (d["entry_price"] is not None
        and prix >= d["entry_price"] + self._atr_tp_mult * atr)

    if stop_touche or tp_touche:
        self.liquidate(symbol)
        d["entry_price"] = None
        d["stop_price"] = None
        continue # une position par actif : on ne réévalue pas l'entrée

# --- Logique d'entrée ---
regime_haussier = prix > d["trend"].current.value
momentum = (d["macd"].current.value > d["macd"].signal.current.value
    and d["rsi"].current.value > 50)

if regime_haussier and momentum:
    # Plafonnement du nombre de positions ouvertes.
    ouvertes = sum(1 for s in self._data if self.portfolio[s].invested)
    if ouvertes >= self._max_positions:
        continue

    # Dimensionnement par Le risque : actions = (risque $) / (distance au stop).
    risque_dollars = self.portfolio.total_portfolio_value * self._risk_per_trade
    distance_stop = self._atr_stop_mult * atr
    if distance_stop <= 0:
        continue

    quantite = int(risque_dollars / distance_stop)
```

```

        if quantite > 0:
            self.market_order(symbol, quantite)
            d["entry_price"] = prix
            d["stop_price"] = prix - distance_stop

def on_end_of_algorithm(self):
    self.log(f"Valeur finale du portefeuille : "
            f"{self.portfolio.total_portfolio_value:,.0f} $")

```

À retenir : cet exemple combine filtre de régime, multi-indicateurs (SMA, MACD, RSI, ATR), dimensionnement par le risque (volatilité), stop suiveur et take-profit dynamiques, et plafonnement du nombre de positions. Le caractère **évènementiel** de LEAN rend naturel le calcul de la taille en fonction de la valeur **courante** du portefeuille et la limite du nombre de positions ouvertes — deux choses plus délicates à exprimer en vectorisé (voir la version Vectorbt du document jumeau).

15. Conclusion

LEAN propose un cadre **évènementiel et réaliste** : chaque exemple ci-dessus simule la prise de décision barre par barre, comme en trading réel. Cette fidélité a un coût — un backtest LEAN est plus lent qu'une approche vectorisée et l'optimisation passe par des exécutions répétées — mais elle se rapproche fortement des conditions de marché et permet de passer en live sans réécrire le code.

Récapitulatif des réflexes acquis :

- Hériter de `QCAgorithm`, configurer dans `initialize`, décider dans `on_data`.
- Utiliser les méthodes d'aide d'indicateurs (`sma`, `rsi`, `bb`, `macd`) et toujours vérifier `is_ready` / `is_warming_up`.
- Passer les ordres via `set_holdings` / `liquidate`.
- Mesurer la performance contre un Buy & Hold, et se méfier du sur-optimisation grâce à la validation hors-échantillon.

Le document jumeau montre comment **les mêmes onze stratégies** s'écrivent de façon **vectorisée** avec Vectorbt / Vectorbt Pro — une comparaison très instructive entre les deux philosophies de backtesting.